

Reviewer Pack — Haar-Wavelet Star-Discrepancy of Sign Matrix Lower-Bounds Ri...

Entry bb503edce1c2 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-04-27 14:18:27 UTC

Haar-Wavelet Star-Discrepancy of Sign Matrix Lower-Bounds Rigidity at Rank $n/2$

Entry ID: bb503edce1c2 **Verdict:** INCONCLUSIVE **Recorded:** 2026-04-27 14:18:27 UTC

1. Conjecture

Field A (mathematical branch): Combinatorial discrepancy via 2D Haar-wavelet star-discrepancy of ± 1 sign matrices (Roth-Schmidt / Matousek L^2 anchored-box discrepancy on the dyadic grid; rarely deployed against rigidity) **Field B** (complexity object): Matrix rigidity (Valiant) at target rank $r = n/2$: minimum Hamming weight $R(M, n/2)$ of a perturbation E with $\text{rank}(M+E) \leq n/2$ over the rationals, the standard pre-Razborov rigidity proxy

Statement:

For every $n \times n$ sign matrix M with n a power of 2, let $D(M) = \max$ over dyadic anchored boxes $B \subseteq [n] \times [n]$ of $|\sum_{(i,j) \in B} M_{ij}| / \sqrt{|B|}$, the normalized Haar star-discrepancy. We conjecture $R(M, n/2) \geq c \cdot D(M)^2 / \log n$ for some absolute $c > 0$, and in particular for the Hadamard sign matrix H_n we conjecture $D(H_n) = \Theta(\sqrt{n})$ so that $R(H_n, n/2) = \Omega(n / \log n)$. One $n \leq 16$ sign matrix M with $\text{rank}(M+E) \leq n/2$ achievable at Hamming weight $< c \cdot D(M)^2 / \log n$ falsifies it.

Rationale (proposer's reasoning):

Star-discrepancy on the dyadic anchored-box system measures how badly a sign pattern aligns with low-rank Haar tensor projectors, and any low-rank perturbation can only destroy $O(\text{rank} \cdot \log n)$ Haar coefficients; thus a large worst-anchored-box imbalance should force many sign flips to reach rank $n/2$. This bridges Roth-Schmidt L^2 discrepancy machinery — never directly fed into Valiant rigidity — to the explicit-Hadamard frontier where existing rank-trick bounds saturate at $\Omega(n^2/r \cdot \log(n/r))$. The constructive mapping (enumerate dyadic boxes, sum entries, normalize) is computable in $O(n^2 \log^2 n)$ and gives a falsifier-friendly quantitative invariant.

Taxonomy category: DISPERSION_DISCREPANCY (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): c4cd5c5f7b5fe205

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

Across 5 seeds $\times \geq 200$ instances per seed (Hadamard, random ± 1 , planted low-rank+sparse, QR-circulant) for $n \in \{4, 8, 16\}$, compute ratio $\rho = R(M, n/2) \cdot \log_2(n) / D(M)^2$. Conjecture SUPPORTED if $\geq 90\%$ of instances per seed have $\rho \geq 0.05$ AND median ρ across all instances ≥ 0.05 ; FALSIFIED if any instance yields $\rho < 0.05$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.90	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.78	SAFE	SAFE
ALGEBRIZATION	SAFE	0.82	SAFE	SAFE
KARP_LIPTON	SAFE	0.86	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 9 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): -matrix rigidity Hadamard discrepancy Haar wavelet sign matrix -Valiant rigidity lower bound anchored box discrepancy dyadic -rank $n/2$ rigidity sign matrix L2 star-discrepancy Roth

Top relevant hits considered: - [http://arxiv.org/abs/cs/0702067v1] The Haar Wavelet Transform of a Dendrogram: Additional Notes - [http://arxiv.org/abs/1005.0979v1] Supersymmetry in Random Matrix Theory - [http://arxiv.org/abs/1112.1588v2] Evaluating Matrix Functions by Resummations on Graphs: the Method of Path-Sums - [http://arxiv.org/abs/0909.2030v2] Size Bounds for Conjunctive Queries with General Functional Dependencies - [http://arxiv.org/abs/1308.3946v1] Optimal Algorithms for Testing Closeness of Discrete Distributions - [http://arxiv.org/abs/0906.0693v3] An improved lower bound on the counterfeit coins problem - [http://arxiv.org/abs/1306.2976v2] Seshadri constants, Diophantine approximation, and Roth's Theorem for arbitrary varieties - [http://arxiv.org/abs/2411.10363v3] Hammersley Point Sets and Inverse of Star-Discrepancy

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, $\leq 90s$ wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
import sys
import json

def dot_product(a, b):
    return sum(x * y for x, y in zip(a, b))

def matrix_multiply(A, B):
    m = len(A)
    n = len(B[0])
    p = len(B)
    C = [[0] * n for _ in range(m)]
    for i in range(m):
        for j in range(n):
```

```

        for k in range(p):
            C[i][j] += A[i][k] * B[k][j]
    return C

def hadamard_matrix(n):
    if n == 1:
        return [[1]]
    H = hadamard_matrix(n // 2)
    top_left = H
    top_right = H
    bottom_left = H
    bottom_right = [-x for x in H]
    return [
        [a + b for a, b in zip(top_left, top_right)],
        [a - b for a, b in zip(bottom_left, bottom_right)]
    ]

def rank(matrix):
    n = len(matrix)
    m = len(matrix[0])
    A = [[matrix[i][j] for j in range(m)] for i in range(n)]
    pivot_row = 0
    pivot_col = 0
    while pivot_row < n and pivot_col < m:
        if matrix[pivot_row][pivot_col] == 0:
            swap_found = False
            for i in range(pivot_row + 1, n):
                if matrix[i][pivot_col] != 0:
                    matrix[pivot_row], matrix[i] = matrix[i], matrix[pivot_row]
                    swap_found = True
                    break
            if not swap_found:
                pivot_col += 1
                continue
        for i in range(pivot_row + 1, n):
            factor = -matrix[i][pivot_col] / matrix[pivot_row][pivot_col]
            for j in range(m):
                matrix[i][j] += factor * matrix[pivot_row][j]
        pivot_row += 1
        pivot_col += 1
    return min(pivot_row, m)

def dyadic_discrepancy(M, n):
    max_discrepancy = 0
    for i in range(n):
        for j in range(n):
            box_sum = sum(M[x][y] for x in range(i, min(i + (1 << int(math.log2(n))), n)) for y in range(j, min(j + (1 << int(math.log2(n))), n)))
            discrepancy = abs(box_sum) / math.sqrt((1 << int(math.log2(n))) ** 2)
            if discrepancy > max_discrepancy:
                max_discrepancy = discrepancy
    return max_discrepancy

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n_values = [4, 8, 16]
    results = []

```

```

for n in n_values:
    H_n = hadamard_matrix(n)
    D_H_n = dyadic_discrepancy(H_n, n)
    R_H_n_n2 = rank(H_n + [[random.choice([-1, 1]) for _ in range(n)] for _ in range(int(n / 2))])

    results.append({
        "n": n,
        "D_H_n": D_H_n,
        "R_H_n_n2": R_H_n_n2
    })

return {
    "metric_name": "R(M, n/2) * log(n) / D^2",
    "metric_value": sum(result["R_H_n_n2"] * math.log(result["n"]) / result["D_H_n"] ** 2 for result in results),
    "instances_tested": len(results),
    "conjecture_holds": all(result["R_H_n_n2"] * math.log(result["n"]) / result["D_H_n"] ** 2 < 1 for result in results),
    "counterexample": ""
}

if __name__ == "__main__":
    seeds = [int(s) for s in sys.argv[1:]] if sys.argv[1:] else [11, 23, 37, 53, 71]

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {json.dumps(result)}")

    # Compute mean/std of metric_value, fraction of seeds where conjecture_holds
    all_results = [run_trial(seed) for seed in seeds]
    mean_metric_value = sum(result["metric_value"] for result in all_results) / len(all_results)
    std_metric_value = math.sqrt(sum((result["metric_value"] - mean_metric_value) ** 2 for result in all_results) / len(all_results))
    support_fraction = sum(1 for result in all_results if result["conjecture_holds"]) / len(all_results)

    if support_fraction >= 0.9:
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction={support_fraction}")
    else:
        first_failing_seed = next(seed for seed, result in enumerate(all_results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"first failing seed\" first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_63ec3b34.py", line 108, in <module>
    result = run_trial(seed)
    ~~~~~^~~~~~

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_63ec3b34.py", line 86, in run_trial
    H_n = hadamard_matrix(n)
    ~~~~~^~~~~~

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_63ec3b34.py", line 35, in hadamard_matrix
    H = hadamard_matrix(n // 2)

```

^^

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_63ec3b34.py", line 39, in hadamard_matrix
    bottom_right = [-x for x in H]
                   ^^
```

TypeError: bad operand type for unary -: 'list'

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed with a `TypeError` in `hadamard_matrix` (negating a list of lists instead of elements), so no instances were evaluated and neither the support nor falsification condition can be assessed. | next: Fix `hadamard_matrix` to recurse on `numpy` arrays (or negate element-wise) and rerun the full 5-seed $\times \geq 200$ -instance protocol for $n \in \{4,8,16\}$.

11. Audit log (LLM calls)

Total LLM calls: 7

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	claude_max	opus	0	0	21074
2	preregistration	claude_max	opus	0	0	7879
3	novelty	claude_max	opus	0	0	3604
4	novelty	claude_max	opus	0	0	9565
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12861
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13247
7	judge	claude_max	opus	0	0	5461

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 73691 ms total latency. Provider mix: {'claude_max': 5, 'ollama_remote': 2}

(full prompt+response transcripts available in `research/audit/bb503edce1c2.jsonl`)

12. Reproducibility

To reproduce this cycle:

- 1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
- 2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
- 3. The full LLM transcripts are in `research/audit/bb503edce1c2.jsonl` for verification of provenance
- 4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/bb503edce1c2.tar.gz` (if generated)